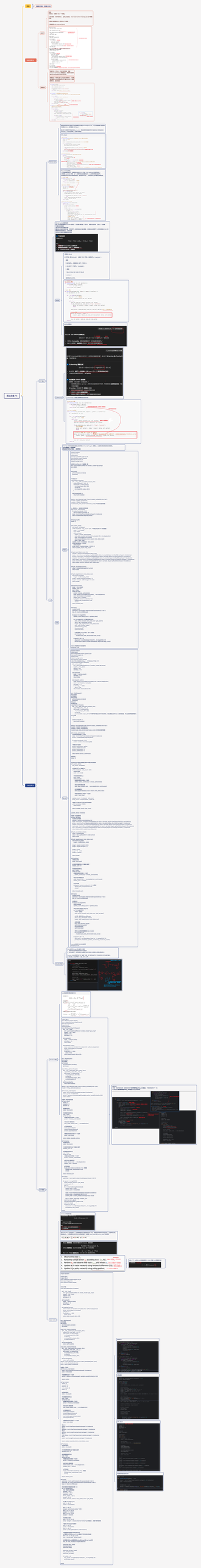
第五章 Ray
本章学习目标：掌握Ray分布式计算框架，能够独立设计并实现大规模分布式AI应用系统
Ray核心概念与架构
Ray的设计哲学与核心抽象
Ray的设计哲学：简单、通用、高性能
Ray的核心抽象：Task（无状态计算）、Actor（有状态计算）、Object（分布式内存对象）
Ray的分布式调度与内存管理
Ray的分布式调度：GCS（全局控制服务）、Raylet（本地调度器）、Object Store（共享内存对象存储）
Ray的内存管理：引用计数、对象溢出、内存监控
Ray Actor模型与状态管理
Actor模型：每个Actor是一个独立的计算单元，拥有自己的状态，通过消息传递通信，天然适合并发
Actor的生命周期：创建、调度、执行、重启、销毁
Actor的容错：检查点、重启策略、状态恢复
Ray Serve模型服务
Ray Serve的设计：基于Actor的模型服务框架，支持HTTP/gRPC接口，内置负载均衡、自动扩缩容、版本管理
Ray Serve的生产特性：健康检查、优雅关闭、流量切换、A/B测试、灰度发布
Ray Serve的性能：单节点吞吐可达数万QPS，延迟开销约1ms，适合实时推理场景
Ray Serve的部署：Kubernetes Operator、Helm Chart、与Ray Cluster集成
Ray Tune超参数调优
Ray Tune的设计：支持多种搜索算法（Grid、Random、Bayesian、Hyperband、ASHA）、多种调度算法（FIFO、ASHA、PBT）、与主流ML框架集成
Ray Tune的实现：Trial管理、结果收集、检查点保存、早停机制、资源分配
Ray Tune的扩展：自定义搜索算法、自定义调度器、自定义回调、与MLflow/W&B集成
Ray Data数据处理
Ray Data的设计：基于Arrow的分布式数据集、支持Map/Filter/Group/Join等算子、与Spark/Dask对比
Ray Data的实现：Block管理、流式执行、内存管理、与Parquet/CSV/JSON/Image数据源集成
Ray Data的应用：大规模数据预处理、特征工程、数据加载、与训练流水线集成
Ray Train分布式训练
Ray Train的设计：支持PyTorch/TensorFlow/XGBoost/LightGBM/Horovod、与Ray Tune集成、支持弹性训练
Ray Train的实现：Worker组管理、梯度同步、检查点保存、故障恢复、弹性扩缩容
Ray Train的性能：与原生PyTorch DDP性能对比、通信开销分析、大模型训练优化
Ray Cluster集群管理
Ray Cluster的设计：Head Node、Worker Node、自动扩缩容、资源调度、与Kubernetes集成
Ray Cluster的实现：Autoscaler、Ray Operator、Helm Chart、Terraform模块、多云部署
Ray Cluster的运维：监控指标、日志收集、告警规则、容量规划、升级策略、灾备方案
Ray Cluster的安全：网络隔离、TLS加密、RBAC权限、审计日志、密钥管理
Ray Cluster的成本：Spot实例利用、资源回收、闲置检测、成本分摊、预算告警
Ray Cluster的实践：生产部署清单、性能调优、故障排查、容量规划、升级迁移、灾备演练
Ray Cluster的工具：Ray Dashboard、Ray CLI、Ray Client、Ray Jobs、Ray Serve Dashboard、Prometheus Exporter、Grafana Dashboard、Alertmanager规则、Terraform模块、Helm Chart、Kubernetes Operator、Ansible Playbook、Docker镜像、CI/CD流水线、自动化测试、性能基准、容量规划模板、升级迁移脚本、灾备演练脚本、运维手册、故障排查手册、SLA定义、SLO定义、SLI定义、错误预算、变更管理、事故管理、容量规划、成本优化、安全加固、合规检查、审计日志、密钥轮换、证书管理、网络隔离、TLS加密、RBAC权限、审计日志、密钥管理